MACHINE LEARNING FOUNDATIONS

Notes on Pytorch (II)

University Master's Degree in
Machine Learning

upna
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Dataset and Dataloader

- Data Management in DL problems is very important
- Datasets used for training are often huge
  - Loading everything into memory is impossible or inefficient
- Training requirements
  - Sequential or random access
  - Splitting data into batches
  - Sometimes, applying transformations to data
  - Efficient, fast and parallel loading
- Solution in PyTorch
  - Dataset: defines how to access de data
  - DataLoader: manages loading of data

Dataset in PyTorch

- Base class torch.utils.data.Dataset
- Abstraction of a set of pairs (x,y)
  - x: data sample
  - y: class or real value associated with x
- When defining custom datasets, we must code two key methods
  - __len__: returns de size of the dataset
  - __getitem__: returns one sample and its label

Datasets in PyTorch

```python
from torch.utils.data import Dataset

class CustomDataset(Dataset):
    def __init__(self, data, labels):
        self.data = data
        self.labels = labels

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return self.data[idx], self.labels[idx]
```

Dataloaders in PyTorch

- Base class: torch.utils.data.DataLoader
- Features
  - Splits datasets into mini-batches
  - Shuffles the data
  - Loads data in paralell
  - Provdides an easy-to-use iterator

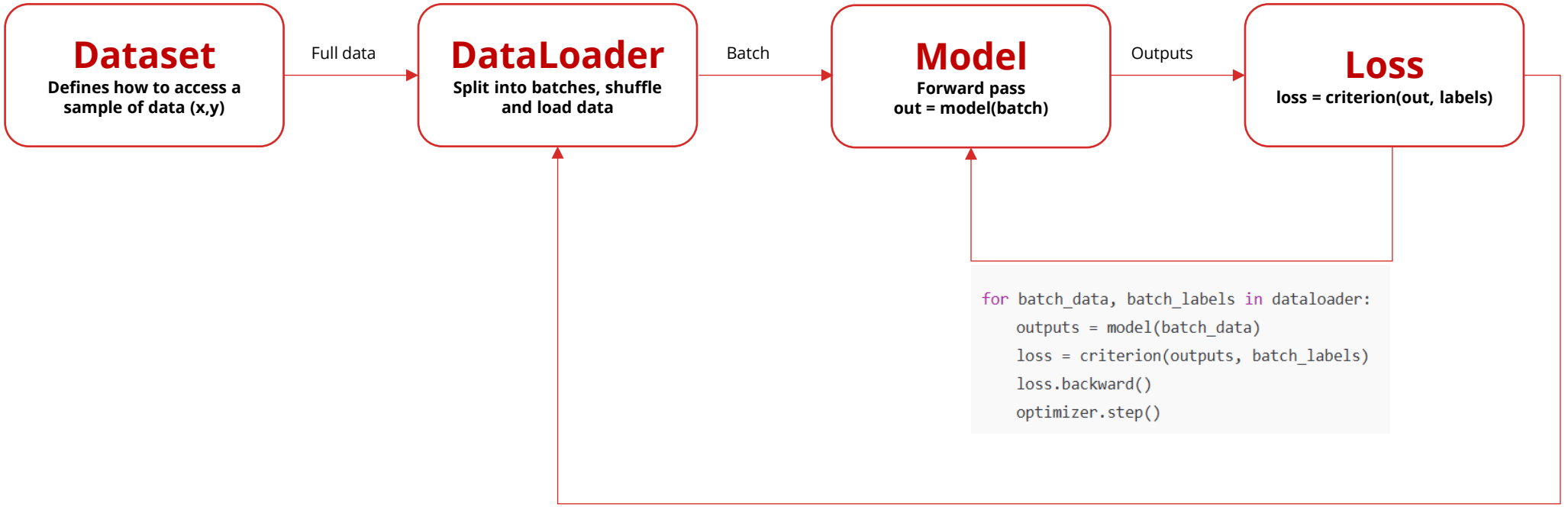
```
from torch.utils.data import DataLoader

dataloader = DataLoader(dataset, batch_size=32, shuffle=True, num_workers=2)
```

Dataflow

- Typical pipeline
  - Dataset: defines data access
  - DataLoader: batches, shuffles and multiprocessing
  - Training loop: model learns from batches
- Two different loops
  - One, looping over the set of batches
  - Another, looping over the number of epochs

Dataflow



For each EPOCH, DataLoader loads again the entire dataset

Transformations

- In many cases, we need to apply one or several transformations to data before sending to the model
  - For example, when working with images, transform the image into a Tensor, resize it, normalize…
- In PyTorch we can define a list of transformations

```
from torchvision import transforms

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
```

Transformations

```python
transform = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Custom dataset
class CustomImageDataset(Dataset):
    def __init__(self, image_paths, labels, transform=None):
        self.image_paths = image_paths
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        img = Image.open(self.image_paths[idx]).convert("RGB")
        label = self.labels[idx]
        if self.transform:
            img = self.transform(img)
        return img, label
```

Train/val/test split

- PyTorch offers data separation for validation using torch.utils.data.random_split
  - Takes PyTorch dataset and allows to split into train/val or train/test with certain distribution
- Usually, only train DataLoader has shuffle = True

```
train_size = int(0.8 * len(ds))
val_size   = len(ds) - train_size
ds_train, ds_val = random_split(ds, [train_size, val_size])

train_loader = DataLoader(ds_train, batch_size=32, shuffle=True)
val_loader   = DataLoader(ds_val,   batch_size=32, shuffle=False)
```

GPU usage in PyTorch

- Now that we are training larger models, we will need the use of GPU to train faster
- To use GPU it is necessary to “move” both the data and the model from CPU to GPU
  - This is done using .to(device)

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Move model
model = MyModel().to(device)

# Inside training loop
for images, labels in train_loader:
    images = images.to(device)
    labels = labels.to(device)

    outputs = model(images)   # Runs on GPU
```

GPU issues

- “Expected all tenors to be on the same device”
  - Model on GPU and data on CPU (or viceversa)
- “CUDA out of memory”
  - Either the model or the batch is too large for GPU memory
  - Try to reduce bath size first